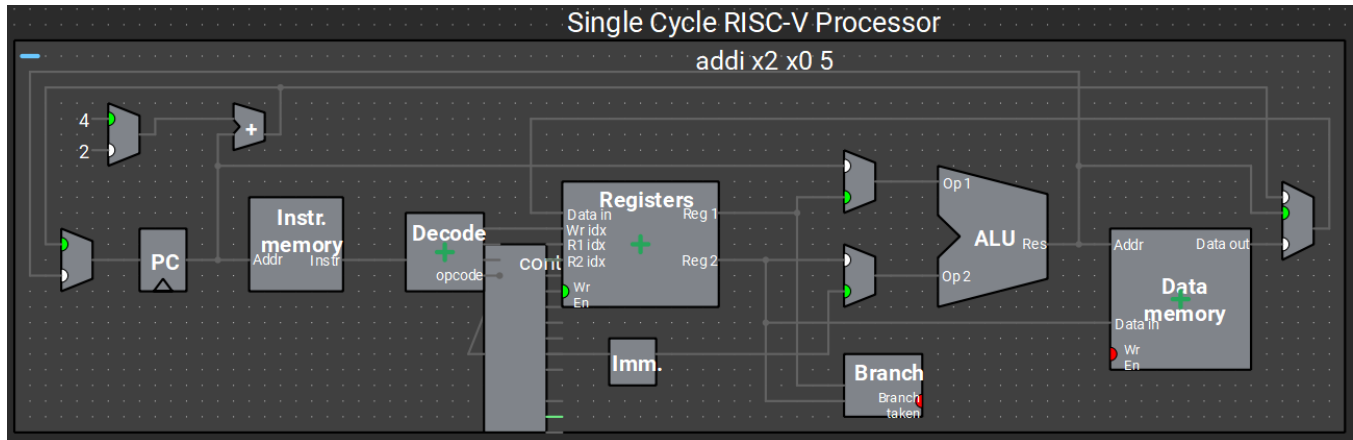


LAB 1: Introduction RIPES

Date: 10 July of 2025



Proffesor: Dr. Francisco Javier Rodríguez Navarrete

Students: Cesar Eduardo Inda Cenicerros

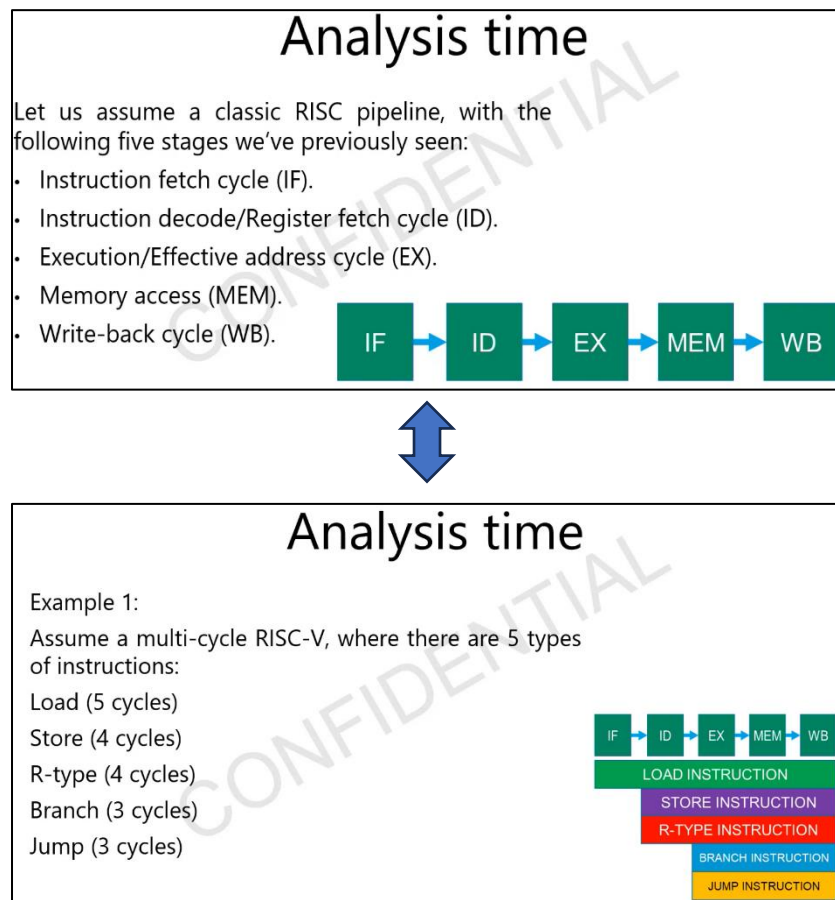
Alonso Emmanuel López Macías

Team: 14

Introduction

Before carrying out this laboratory, we began reviewing general concepts related to Module 6. For this, we defined, for example, what computer architecture is, its main role within a digital circuit, microprocessors, the CPU, the ISA (Instruction Set Architecture), the Von Neumann and Harvard architectures, as well as considerations regarding RISC-V, the comparison between RISC and CISC, RV32I, and many essential concepts needed to start addressing these topics. We also included an introduction to assembly language, how it can be used, and how it can be represented through a simulator such as **"RIPES."**

For this first laboratory, we were asked to extract the tool and start with a test code called **"riscvtest. s"**, which writes the value 25 to address 100. In this case, we relied on the book by Sarah and David Harris for support. We analyzed the behavioral code in the assembly language use in the simulator for test and know about more RISC-V 321 and know how use this tool. We test different cases for use the direction memory and some uses in this tool. Other consideration very important is know about the stages for use the analysis time in the process for the RISC-V.



Requirements

- Compile & test the program in **“Ripes”**.
- Debug **“test the value”**.

Development

We use the example code in the assembly language in the tool.

```
# riscvtest.s
# Sarah.Harris@unlv.edu
# David_Harris@hmc.edu
# 27 Oct 2020
# Test the RISC-V processor:
# add, sub, and, or, slt, addi, lw, sw, beq, jal
# If successful, it should write the value 25 to address 100
# RISC-V Assembly      Description      Address      Machine Code
main:
    addi x2, x0, 5      # x2 = 5      0      00500113
    addi x3, x0, 12     # x3 = 12     4      00C00193
    addi x7, x3, -9     # x7 = (12 - 9) = 3      8      FF718393
    or x4, x7, x2       # x4 = (3 OR 5) = 7      C      0023E233
    and x5, x3, x4      # x5 = (12 AND 7) = 4    10     0041F2B3
    add x5, x5, x4      # x5 = 4 + 7 = 11      14     004282B3
    beq x5, x7, end     # shouldn't be taken    18     02728863
    slt x4, x3, x4      # x4 = (12 < 7) = 0     1C     0041A233
    beq x4, x0, around  # should be taken      20     00020463
    addi x5, x0, 0      # shouldn't execute    24     00000293

around:
    slt x4, x7, x2      # x4 = (3 < 5) = 1      28     0023A233
    add x7, x4, x5      # x7 = (1 + 11) = 12    2C     005203B3
    sub x7, x7, x2      # x7 = (12 - 5) = 7     30     402383B3
    sw x7, 84(x3)       # [96] = 7            34     0471AA23
    lw x2, 96(x0)       # x2 = [96] = 7        38     06002103
    add x9, x2, x5      # x9 = (7 + 11) = 18    3C     005104B3
    jal x3, end         # jump to end, x3 = 0x44 40     008001EF
    addi x2, x0, 1      # shouldn't execute    44     00100113

end:
    add x2, x2, x9      # x2 = (7 + 18) = 25    48     00910133
    sw x2, 0x20(x3)     # [100] = 25           4C     0221A023
```

In the debug we can observe the values as following:

<pre> 3 # David_Harris@hmc.edu 4 # 27 Oct 2020 5 # Test the RISC-V processor: 6 # add, sub, and, or, slt, addi, lw, sw, beq, jal 7 # If successful, it should write the value 25 to address 100 8 # RISC-V Assembly Description Address Machine Code 9 main: 10 addi x2, x0,5 # x2 = 5 0 00500113 11 addi x3, x0, 12 # x3 = 12 4 00c00193 12 addi x7, x3, -9 # x7 = (12 - 9) = 3 8 FF718393 13 or x4, x7, x2 # x4 = (3 OR 5) = 7 C 0023E233 14 and x5, x3, x4 # x5 = (12 AND 7) = 4 10 0041F2B3 15 add x5, x5, x4 # x5 = 4 + 7 = 11 14 004282B3 16 beq x5, x7, end # shouldn't be taken 18 02728863 17 slt x4, x3, x4 # x4 = (12 < 7) = 0 1C 0041A233 18 beq x4, x0, around # should be taken 20 00020463 19 addi x5, x0,0 # shouldn't execute 24 00000293 20 21 around: 22 slt x4, x7, x2 # x4 = (3 < 5) = 1 28 0023A233 23 add x7, x4, x5 # x7 = (1 + 11) = 12 2C 005203B3 24 sub x7, x7, x2 # x7 = (12 - 5) = 7 30 402383B3 25 sw x7, 84(x3) # [96] = 7 34 0471AA23 26 lw x2, 96(x0) # x2 = [96] = 7 38 06002103 27 add x9, x2, x5 # x9 = (7 + 11) = 18 3C 005104B3 28 jal x3, end # jump to end, x3 = 0x44 40 008001EF 29 addi x2, x0,1 # shouldn't execute 44 00100113 30 31 end: 32 add x2, x2, x9 # x2 = (7 + 18) = 25 48 00910133 33 sw x2, 0x20(x3) # [100] = 25 4C 0221a023 </pre>				
<pre> 00000000 <main>: 0: 00500113 addi x2 x0 5 4: 00c00193 addi x3 x0 12 8: ff718393 addi x7 x3 -9 c: 0023e233 or x4 x7 x2 10: 0041f2b3 and x5 x3 x4 14: 004282b3 add x5 x5 x4 18: 02728863 beq x5 x7 48 <end> 1c: 0041a233 slt x4 x3 x4 20: 00020463 beq x4 x0 8 <around> 24: 00000293 addi x5 x0 0 00000028 <around>: 28: 0023a233 slt x4 x7 x2 2c: 005203b3 add x7 x4 x5 30: 402383b3 sub x7 x7 x2 34: 0471aa23 sw x7 84 x3 38: 06002103 lw x2 96 x0 3c: 005104b3 add x9 x2 x5 40: 008001ef jal x3 8 <end> 44: 00100113 addi x2 x0 1 00000048 <end>: 48: 00910133 add x2 x2 x9 4c: 0221a023 sw x2 32 x3 </pre>				

Name	Alias	Value	Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
x0	zero	0x00000000	0x00000030	0x402383b3	0xb3	0x83	0x23	0x40
x1	ra	0x00000000	0x0000002c	0x005203b3	0xb3	0x03	0x52	0x00
x2	sp	0x00000019	0x00000028	0x0023a233	0x33	0xa2	0x23	0x00
x3	gp	0x00000044	0x00000024	0x00000293	0x93	0x02	0x00	0x00
x4	tp	0x00000001	0x00000020	0x00020463	0x63	0x04	0x02	0x00
x5	t0	0x0000000b	0x0000001c	0x0041a233	0x33	0xa2	0x41	0x00
x6	t1	0x00000000	0x00000018	0x02728863	0x63	0x88	0x72	0x02
x7	t2	0x00000007	0x00000014	0x004282b3	0xb3	0x82	0x42	0x00
x8	s0	0x00000000	0x00000010	0x0041f2b3	0xb3	0xf2	0x41	0x00
x9	s1	0x00000012	0x0000000c	0x0023e233	0x33	0xe2	0x23	0x00
			0x00000008	0xff718393	0x93	0x83	0x71	0xff
			0x00000004	0x00c00193	0x93	0x01	0xc0	0x00
			0x00000000	0x00500113	0x13	0x01	0x50	0x00

GitHub Repository

https://github.com/CeicGitHub/Fundamentals Of Digital Design FPGA 6/tree/Lab1_Ripes

Conclusion / Issues

Alonso Emmanuel Lopez Macias

This practice served as a reminder of what assembly language is, as well as an introduction to this programming and simulation tool. I realized that I hadn't perceived many aspects of assembly language in this way before, and the class, along with the lab, helped me understand the great importance of knowing how the internal architecture of a digital circuit works at this level. It also showed me how its components operate, allowing multiple data and information to travel through pulses or signals, and how, through such mechanisms, complex logic can be executed in extremely short periods of time to perform thousands of operations.

Cesar Eduardo Inda Cenicerros

Through this laboratory, I was able to gain a deeper understanding of processor architecture, how a computer works at a more complex level, and its thousands of internal components. I also learned more about RISC-V and its importance in assembly language, as it defines many of the elements responsible for proper programming to observe the behavior of a digital circuit or the architecture of a system. These elements allow for a greater understanding of the complexity involved in operating devices such as memory, an ALU, or logic gates, which are part of our daily lives in modern electronics.